



Spring Framework

Training Assignment

Document Code	25e-BM/HR/HDCV/FSOFT
Version	1.1
Effective Date	20/11/2020

RECORD OF CHANGES

No	Effective Date	Change Description	Reason	Reviewer	Approver
1	26/Sep/2024	Create a new assignment	Create new		

Contents

Unit 8: Spring Validation	4
Objectives:	4
Problem Description:	4
Technical Requirements:	6

 	CODE	:	JSFW_S_A101
	TYPE	:	SHORT
	LOC	:	N/A
	DURATION	:	120 minutes

Unit 8: Spring Validation

Objectives:

After completing this assignment, trainees will:

1. Understand and practice validation concepts in Spring Framework.
2. Use annotations for validating data in a Spring Boot application.
3. Implement form-based validation and handle validation errors in Thymeleaf templates.

Problem Description:

We are beginning to build a new Learning Management System (LMS). This system will eventually consist of microservices for **course sharing** and **student assessment**. In this assignment, we will create the core Course entity for the course-sharing system and implement validation for the "New Course" creation form.

Task 1: Create a Course Entity with Validation

Create a class named Course.java in the model package that represents a course in the LMS:

Contains the following attributes:

1. String title
2. String instructorName
3. String instructorEmail
4. String description
5. Integer durationHours

Add validation annotations to the class to enforce the following rules:

- The title field must not be empty and should have at least 5 characters.
- The instructorName field must not be empty and should have at least 2 characters.
- The instructorEmail field must be a valid email address and must not be empty.
- The description field must not be empty and must be between 10 and 200 characters long.
- The durationHours field must not be null and must be at least 1.

Hint: Use Spring's javax.validation.constraints annotations like **@NotEmpty**, **@Size**, **@Email**, **@Min**, and **@NotNull**.

Task 2: Create a "New Course" Form

Create a Thymeleaf template named `create_course.html` that displays a registration form with fields for title, instructorName, instructorEmail, description, and durationHours.

Use Thymeleaf form tags to bind the form fields to the Course model object.

Include error messages for each field. When a validation error occurs, display the error message next to the corresponding input field.

Example template snippet for the title field:

```
<form action="#" th:action="@{/courses/create}" th:object="${course}"
method="post">
  <div>
    <label for="title">Course Title:</label>
    <input type="text" id="title" th:field="*{title}" />
    <div th:if="${#fields.hasErrors('title')}"
        th:errors="*{title}"
        style="color: red;"></div>
  </div>

  <!-- Add other form fields similarly -->

  <button type="submit">Create Course</button>
</form>
```

Task 3: Create a Controller for Handling Course Creation

Create a controller class named `CourseController.java` in the controller package.

This controller should:

1. Serve the "New Course" page via a GET request to `/courses/new`. (This method should add a new `Course()` object to the model).
2. Handle form submission via a POST request to `/courses/create`.
3. Validate the Course object upon form submission using Spring's `@Valid` annotation.
4. If validation fails, return the user to the `create_course.html` form with validation error messages.
5. If validation is successful, redirect the user to a success page.

Hint: Use `@Valid` for validation and `BindingResult` to check for errors.

Task 4: Create a Success Page

- Create a Thymeleaf template named `create_success.html` that displays a success message after the user successfully creates a course.

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org" (http://www.thymeleaf.org)">
<head>
  <title>Creation Success</title>
```

```
</head>
<body>
  <h2>Course Created Successfully!</h2>
  <p>Thank you for adding a new course to the LMS.</p>
</body>
</html>
```

Technical Requirements:

- **JDK 1.8+**
- **Spring Boot 2+**
- **Maven 3+**
- **Thymeleaf**

-- THE END --